

ISAAC SIM 4.5.0 AMR SOP

This SOP will cover what Isaac Sim is, how to install it, the basics of it, and current progress. The SOP specifically covers how to setup and uses Isaac for the Advantech AMR rover. Throughout this SOP, there will be links to Nvidia resources that will give you more detailed instructions on certain topics if needed.

Index

1. What is Isaac Sim? And Installation Guide (Page 2-4)
2. Isaac Sim basics (Page 5)
3. AMR Setup (Page 6-18)

Current Setup:

For AMR testing, Isaac Sim version 4.5.0 is being used on a Windows 11 computer. Another computer ran on Ubuntu 22.04 has ROS2 Humble Installed. These computers communicate with each other through the ROS2 bridge on Isaac Sim

Part 1: What is Isaac Sim? And Installation Guide

Isaac Sim is **NVIDIA's robotics simulation platform** built on top of **Omniverse**. Think of it as a **virtual testing ground for robots** where you can safely develop, train, and test without needing the real hardware. Using Isaac, we can easily simulate and test our rover in many different environments to see if it works properly. We can use the results from

these tests to then develop and train our robot. Isaac Sim eliminates the need for the physical rover for testing, making it much more efficient to develop the AMR.

Nvidia main website for Isaac Sim: [Isaac Sim Nvidia](#)

Installation guide: [Installation Guide](#)

To download Isaac Sim, your PC will need to have sufficient hardware to properly run it. Through testing, I have found that the program will still work on computers with lower than minimum specs, but the program will be more laggy. I would also recommend to allocate at least 50 to 100 gbs of storage for Isaac Sim and its assets.

System Requirements

Element	Minimum Spec	Good	Ideal
OS	Ubuntu 20.04/22.04 Windows 10/11	Ubuntu 20.04/22.04 Windows 10/11	Ubuntu 20.04/22.04 Windows 10/11
CPU	Intel Core i7 (7th Generation) AMD Ryzen 5	Intel Core i7 (9th Generation) AMD Ryzen 7	Intel Core i9, X-series or higher AMD Ryzen 9, Threadripper or higher
Cores	4	8	16
RAM	32GB [1]	64GB [1]	64GB [1]
Storage	50GB SSD	500GB SSD	1TB NVMe SSD
GPU	GeForce RTX 3070	GeForce RTX 4080	RTX Ada 6000
VRAM	8GB [1] [2]	16GB	48GB

You also need to install the correct drivers based on what OS you plan to run Isaac Sim on. On this tutorial, we will teach how to run Isaac Sim on windows, so 537.58 was installed.

Driver Requirements

The recommended driver version for Isaac Sim is **537.58** for Windows and **535.129.03** for Linux.

Driver Version Support	Windows	Linux
Recommended	537.58 (GameReady, Studio), 537.70 (RTX/Quadro, Grid/vGPU)	535.129.03 (GameReady, Studio, RTX/Quadro, Grid/vGPU)
Minimum	537.58 (GameReady, Studio), 537.70 (RTX/Quadro, Grid/vGPU)	535.129.03 (GameReady, Studio, RTX/Quadro, Grid/vGPU)

After these requirements are met, you can install Isaac Sim. This tutorial will cover how to download version “4.5.0” as that is the latest version out. You should be able to use this tutorial for different versions however some things might change.

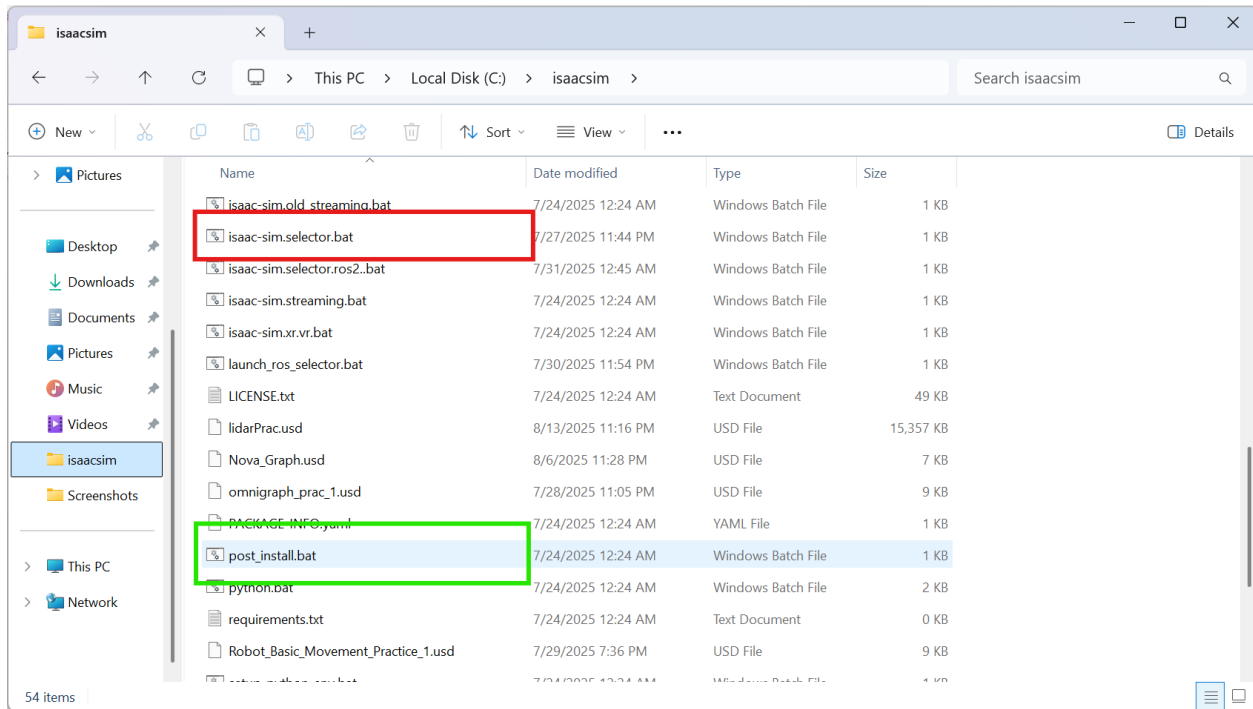
Download Isaac Sim from this link: [Isaac Sim Installation](#)

Latest Release

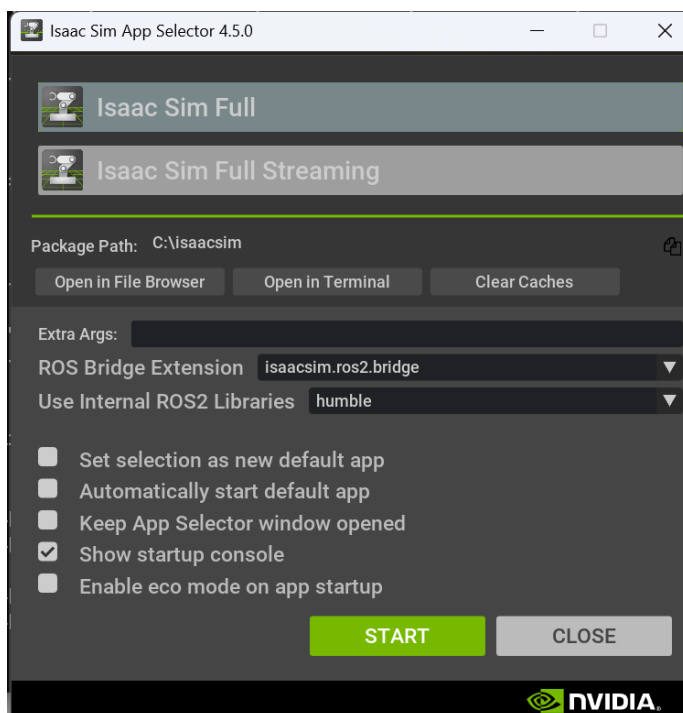
Latest Release

Name	Version	Release Date	Links
Isaac Sim	4.5.0	January 2025	Linux (6.7 GB)
			Windows (6.5 GB)

After installation, there should be a dedicated Isaac Sim folder on your computer. In this folder, first run “post_install.bat” by double clicking it. After this, you can open Isaac Sim by double clicking “isaac-sim.selector.bat”. This should open a terminal, allow it some time to load and it should open a pop up when ready.



On the startup menu, change all the settings to the ones listed here. This will allow you to use the ROS2 bridge.



Part 2: Isaac Sim basics

This section will cover the basics of Isaac Sim. Completing this section will help you understand the program in more depth and allow you to start new projects on your own.

[Isaac Sim 4.5.0 Documentation](#)

If you have any questions or don't understand a concept, you can always refer to the main website or ask ChatGPT.

1) [Workflow](#)

A workflow is the way you choose to interact with and use Isaac Sim. The three main workflows are GUI, extension, and stand-alone python. This SOP covers how to use the GUI as in my opinion it is the easiest to understand and pickup.

2) [Environment Setup](#)

For each project you do, you will want to set up the environment beforehand. This is needed for 3d simulation and pretty much every project you do.

3) [Adding objects, Building robots, Adding cameras & sensors](#)

For the application of setting up the AMR in Isaac Sim, the middle tutorial is not as useful. However, you should still complete the 1st and 3rd tutorial to learn the basics of placing objects and cameras.

4) [Omnigraph](#)

Omnigraph is the visual scripting language used in the GUI. Making action graphs allow you to control and move an assembled robot in any way you like. Here are the resources I recommend looking over to learn Omnigraph

1. [Core Concepts](#) : teaches you the basics
2. [Omnigraph Tutorial](#) : basic tutorial for common Isaac Sim objects
3. [Omnigraph Tutorial 2](#) : teaches you how to use Omnigraph specifically for rovers

For the purpose of the AMR, Omnigraph shortcuts are mainly used. These will show up later in the tutorial.

5) [ROS2](#)

This page shows you how to use Isaac Sim with ROS2.

I recommend following these two tutorials: [URDF Import](#), [ROS2 Drive Messages](#).

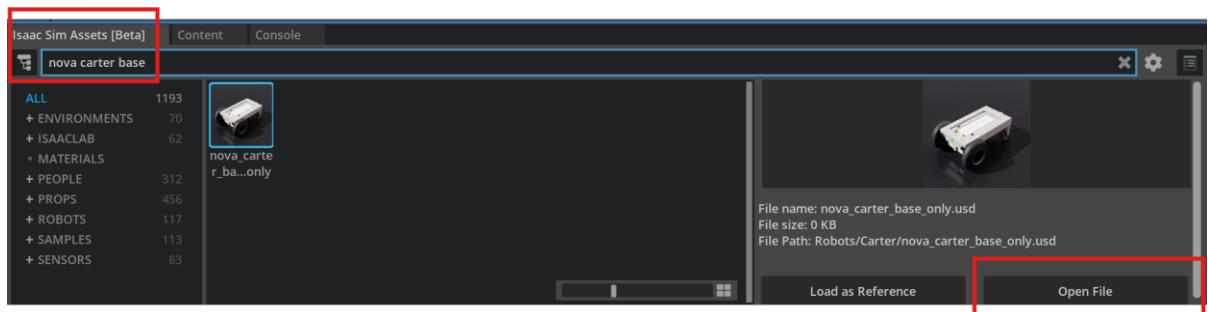
Also view these pages to understand more about how ROS2 works with Isaac Sim: [ROS2 Clock](#), [ROS2 RTF](#).

These tutorials are not necessarily needed, but you can refer to them if you don't understand something later in the SOP: [ROS2 Camera](#), [ROS2 Lidar](#).

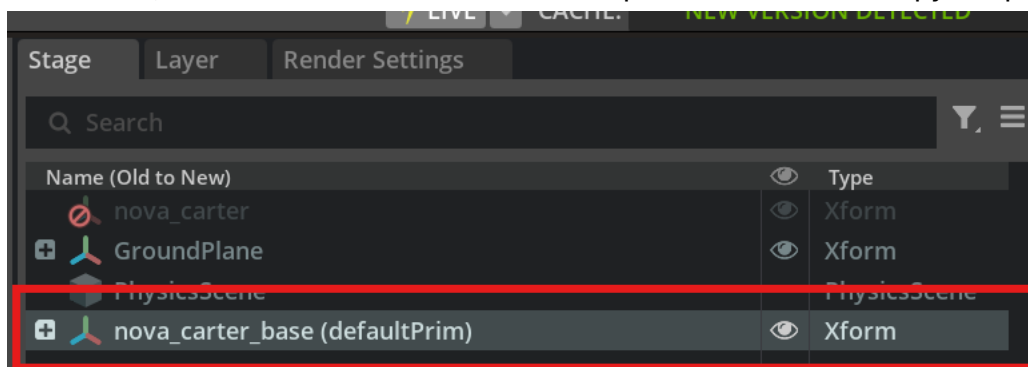
Part 3: AMR Setup

This section will cover how to setup the AMR in a virtual environment from scratch

- 1) Open Isaac Sim. Press “file -> new” to create a new project. Add a physics scene and a ground plane (This is the setup for a basic barebone environment).
- 2) As mentioned earlier in the SOP, URDF files are basically the 3d model and joint configuration of a specific robot. Since there is not a URDF file provided by Agile (The company that made our rover: the agile mini), we will have to borrow a rover asset from Isaac Sim. Before we do this, save the current stage as “AMR”. On the bottom left, type in “nova carter base” and make sure to press “open file”. The nova carter is a good substitute because just like our AMR, it has two front wheels for drive, and two caster wheels in the back that freely rotate.

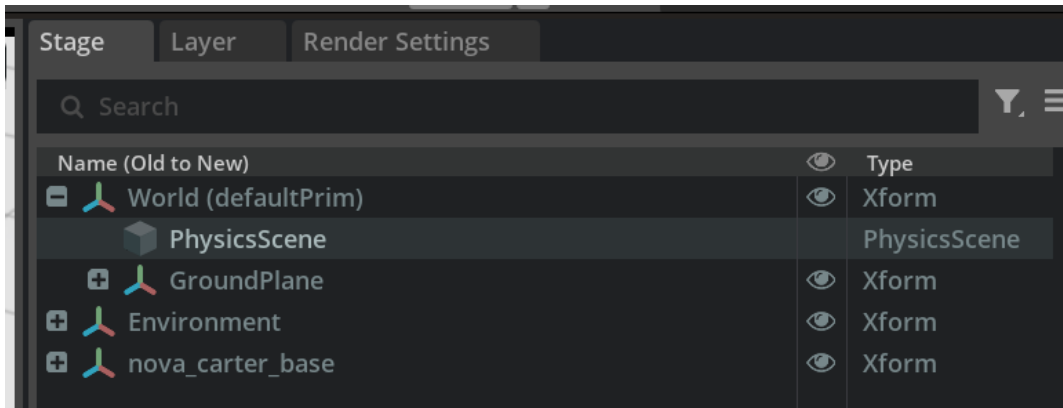


This will open a whole new scene but don't worry. After this, go to the right side of the screen, click on "nova_carter_base" and press "Ctrl + C" to copy the prim.

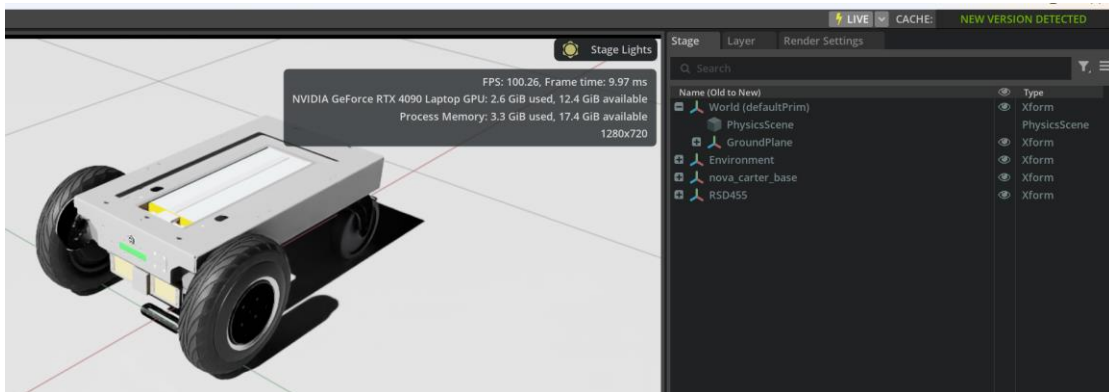


After this you can reopen "AMR" by pressing "file -> open" then press "Ctrl + V" (IN THE STAGE ON THE RIGHT SIDE) to paste it. Your stage should then end up looking

like this

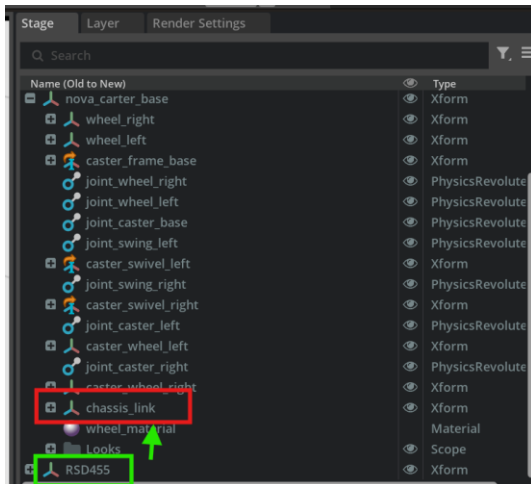


- 3) Once we have placed our rover, we can now add the camera and lidar. First, we will add the camera. Please save the file before you do this. After, go back to “Isaac Sim Assets” on the bottom left and type in “rsd455”. Repeat the process of opening the file, copying it, and pasting it in the AMR project. If you correctly do this, your screen should now look like this.

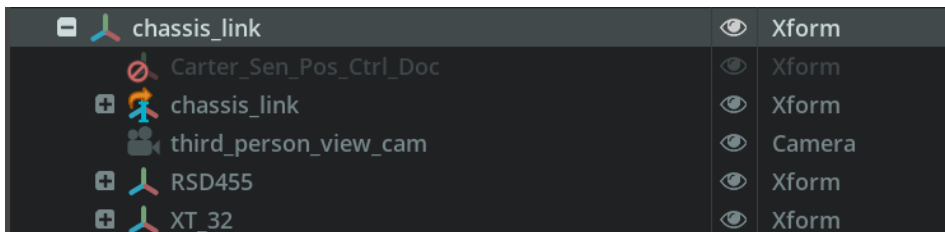


After this, expand “nova carter base” on the stage and drag “RSD455” under “chassis link”. This step is very important as it physically connects the camera to

the rover so when the rover moves, the camera moves with it.

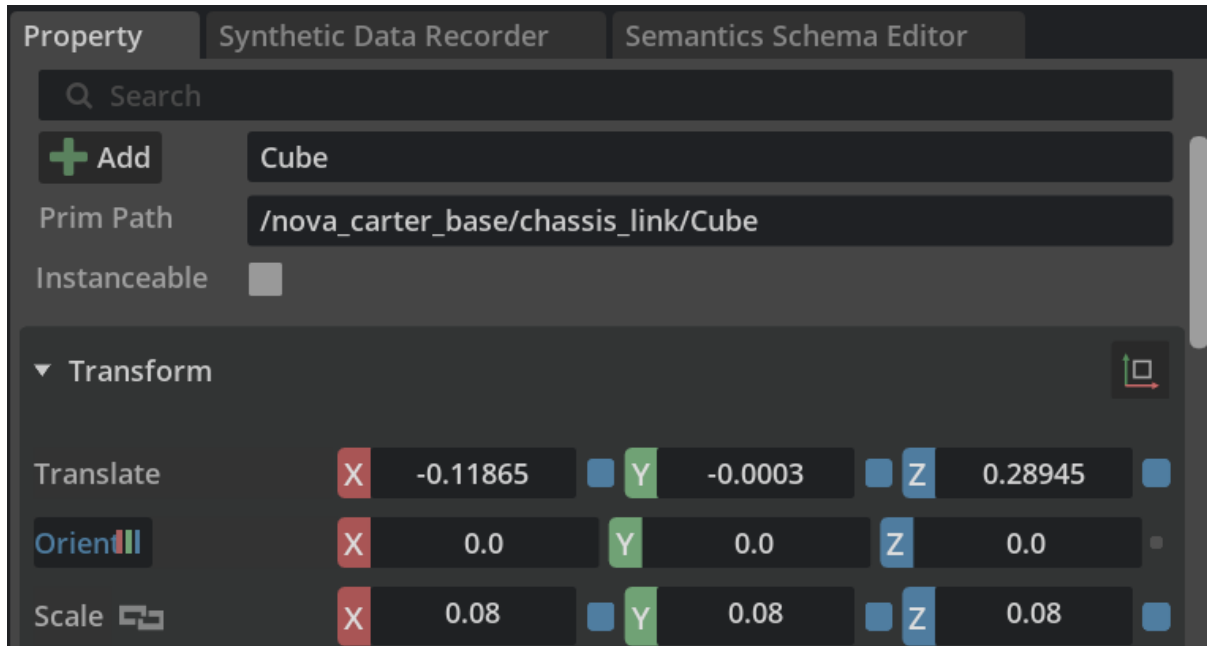


Next, do the same process for the “XT-32” (Open it in assets, add under chassis link). Your chassis link root should now look like this

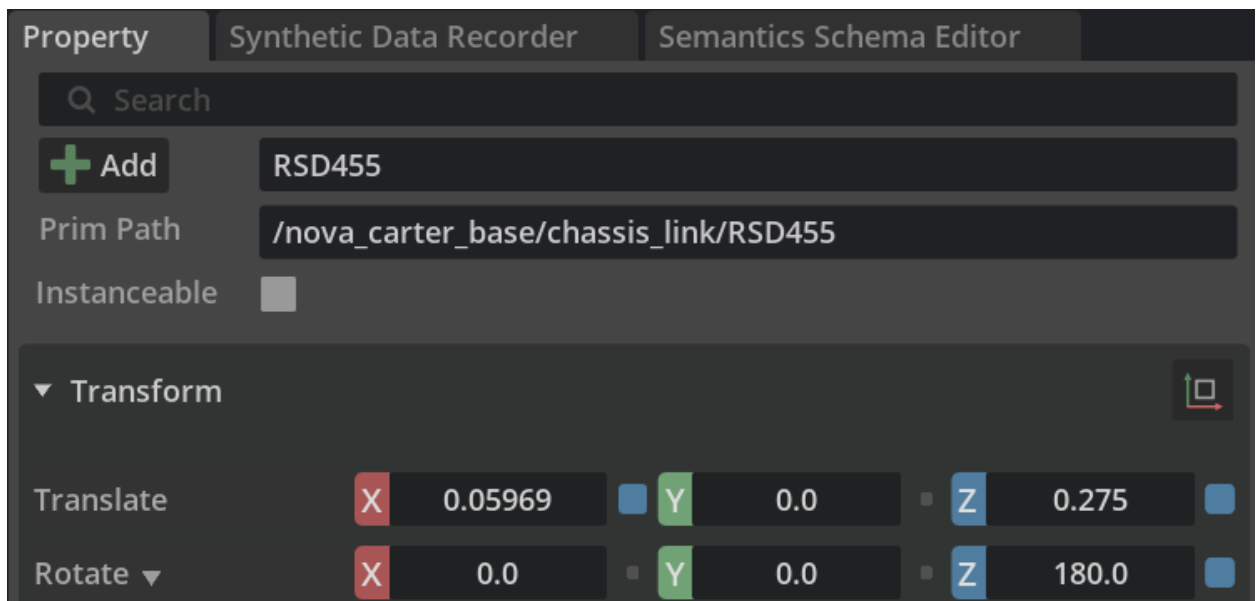


- 4) We will now adjust the position of these items, so they are in the right place on the rover. First let's add a cube to the rover. Press “create -> mesh -> cube” and place the cube under chassis link like before. After this, go to properties and change the

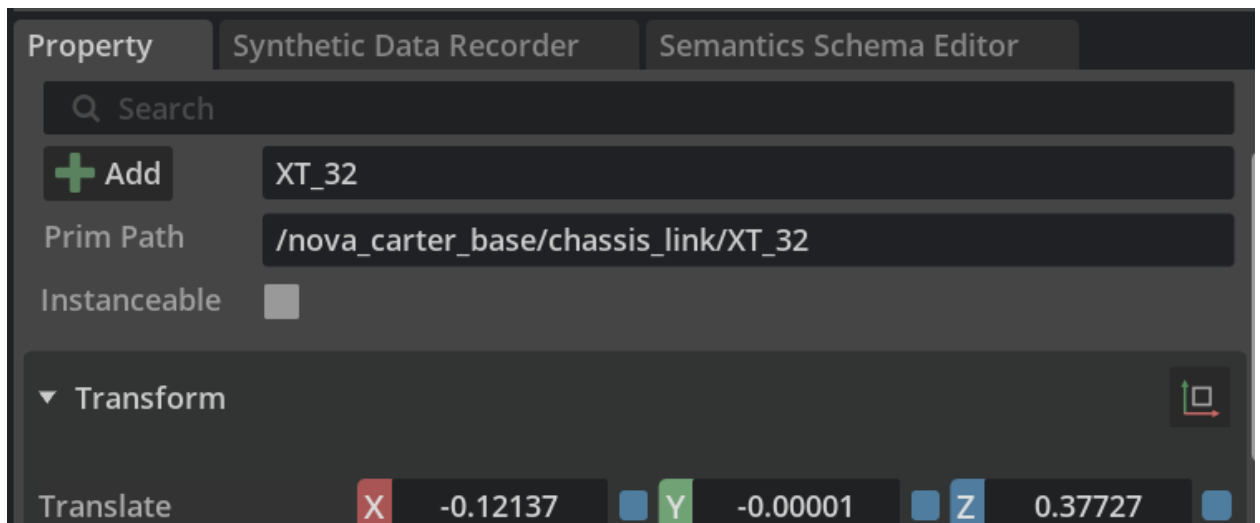
settings to the one below.



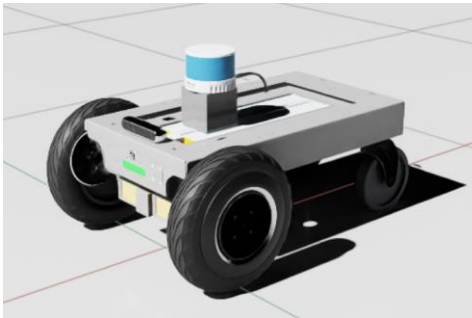
Change the camera settings to this.



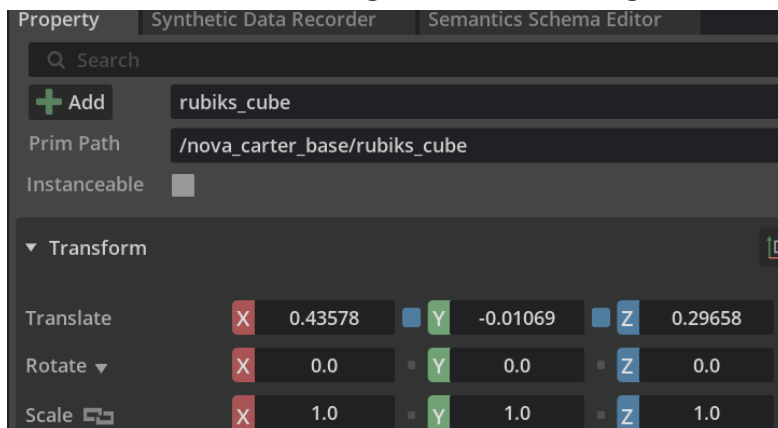
Change the Lidar settings to this.



Your rover should now look like this

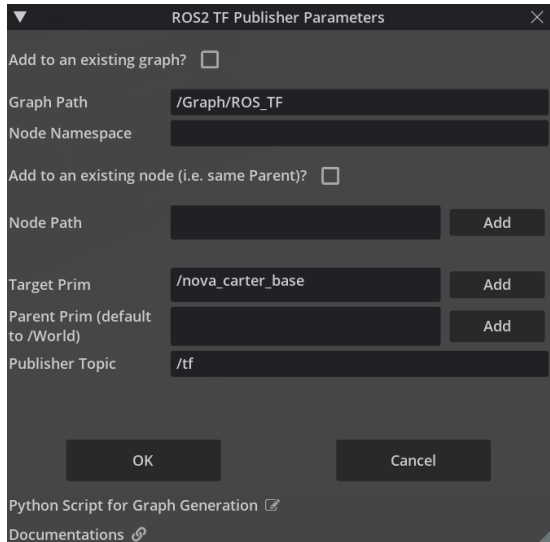


The last step is going to the assets, typing “rubiks cube” and loading it as a reference. After this, change it to these settings



To adjust the camera, use “Alt + left click” “Alt + right click” and “right click”

- 5) We will now add the action graphs for all the components of the rover. To add action graphs, go to the toolbar on the top left and press “Tools -> Robotics -> ROS 2 Omnigraphs”. First add the “clock” graph. Then add the “TF Publisher” graph with these settings and press ok.



The image shows the 'ROS2 TF Publisher Parameters' dialog box. It has a title bar with a close button. The main area contains several fields and checkboxes. At the top, there is a checkbox 'Add to an existing graph?' which is unchecked. Below it is a 'Graph Path' field with the value '/Graph/ROS_TF'. Underneath is a 'Node Namespace' field. Then another checkbox 'Add to an existing node (i.e. same Parent)?' is unchecked. Below this is a 'Node Path' field with an 'Add' button next to it. Then a 'Target Prim' field with the value '/nova_carter_base' and an 'Add' button. Below that is a 'Parent Prim (default to /World)' field with an 'Add' button. At the bottom of the main area is a 'Publisher Topic' field with the value '/tf'. At the very bottom are 'OK' and 'Cancel' buttons. Below the buttons, there is a link 'Python Script for Graph Generation' and a link 'Documentations'.

ROS2 TF Publisher Parameters

Add to an existing graph? ☐

Graph Path: /Graph/ROS_TF

Node Namespace:

Add to an existing node (i.e. same Parent)? ☐

Node Path: Add

Target Prim: /nova_carter_base Add

Parent Prim (default to /World): Add

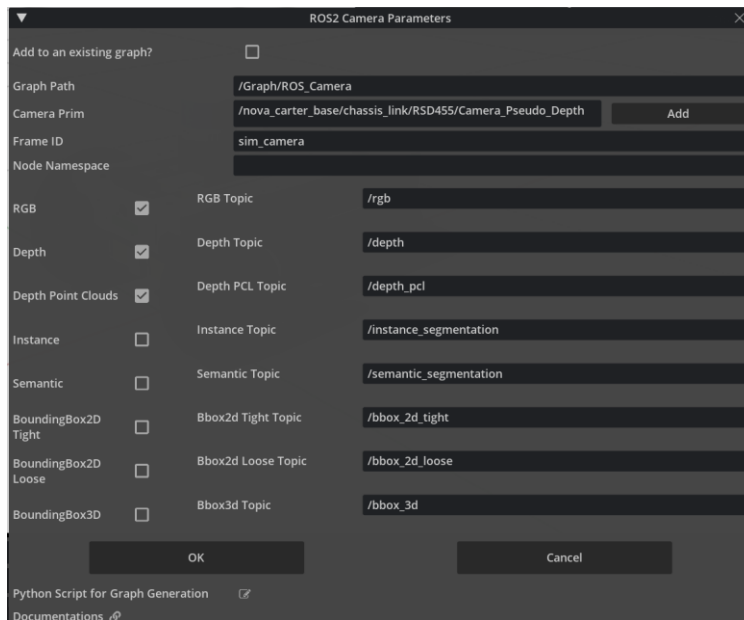
Publisher Topic: /tf

OK Cancel

Python Script for Graph Generation [Python Script for Graph Generation](#)

Documentations [Documentations](#)

Add the “Camera” graph with these settings.



The image shows the 'ROS2 Camera Parameters' dialog box. It has a title bar with a close button. The main area contains several fields and checkboxes. At the top, there is a checkbox 'Add to an existing graph?' which is unchecked. Below it is a 'Graph Path' field with the value '/Graph/ROS_Camera'. Underneath is a 'Camera Prim' field with the value '/nova_carter_base/chassis_link/RSD455/Camera_Pseudo_Depth' and an 'Add' button. Below that is a 'Frame ID' field with the value 'sim_camera'. Then a 'Node Namespace' field. Below this is a section with checkboxes for 'RGB', 'Depth', 'Depth Point Clouds', 'Instance', 'Semantic', 'BoundingBox2D Tight', 'BoundingBox2D Loose', and 'BoundingBox3D'. To the right of these checkboxes are corresponding topic fields: 'RGB Topic' with '/rgb', 'Depth Topic' with '/depth', 'Depth PCL Topic' with '/depth_pcl', 'Instance Topic' with '/instance_segmentation', 'Semantic Topic' with '/semantic_segmentation', 'BoundingBox2D Tight Topic' with '/bbox_2d_tight', 'BoundingBox2D Loose Topic' with '/bbox_2d_loose', and 'BoundingBox3D Topic' with '/bbox_3d'. At the bottom are 'OK' and 'Cancel' buttons. Below the buttons, there is a link 'Python Script for Graph Generation' and a link 'Documentations'.

ROS2 Camera Parameters

Add to an existing graph? ☐

Graph Path: /Graph/ROS_Camera

Camera Prim: /nova_carter_base/chassis_link/RSD455/Camera_Pseudo_Depth Add

Frame ID: sim_camera

Node Namespace:

RGB ☒ RGB Topic: /rgb

Depth ☒ Depth Topic: /depth

Depth Point Clouds ☒ Depth PCL Topic: /depth_pcl

Instance ☐ Instance Topic: /instance_segmentation

Semantic ☐ Semantic Topic: /semantic_segmentation

BoundingBox2D Tight ☐ BoundingBox2D Tight Topic: /bbox_2d_tight

BoundingBox2D Loose ☐ BoundingBox2D Loose Topic: /bbox_2d_loose

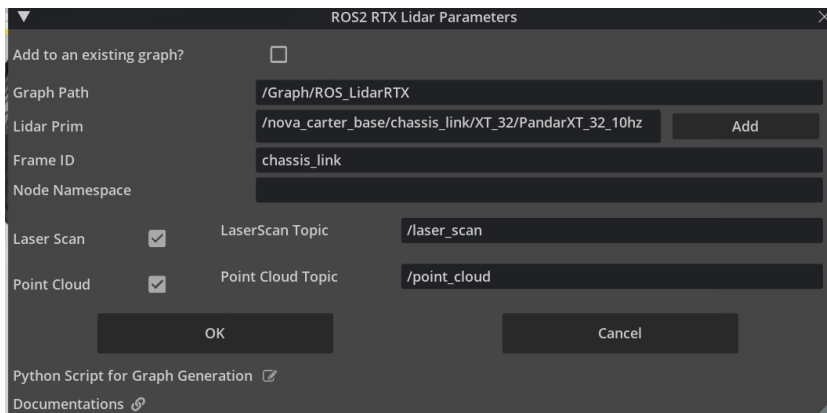
BoundingBox3D ☐ BoundingBox3D Topic: /bbox_3d

OK Cancel

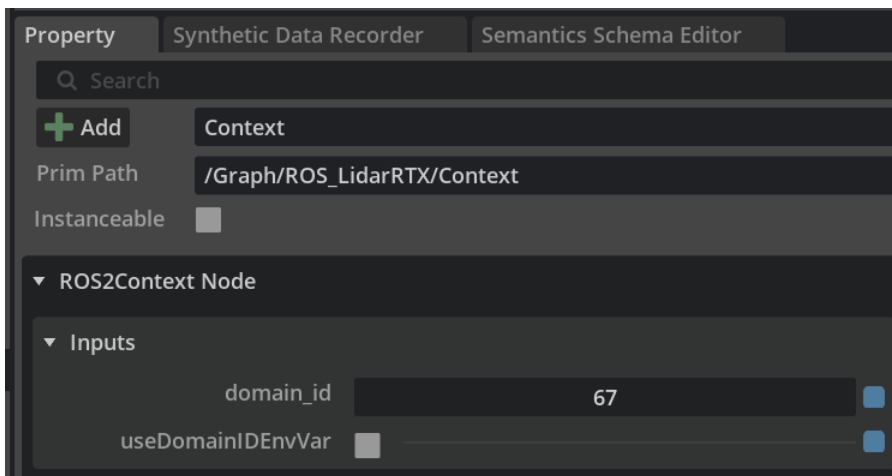
Python Script for Graph Generation [Python Script for Graph Generation](#)

Documentations [Documentations](#)

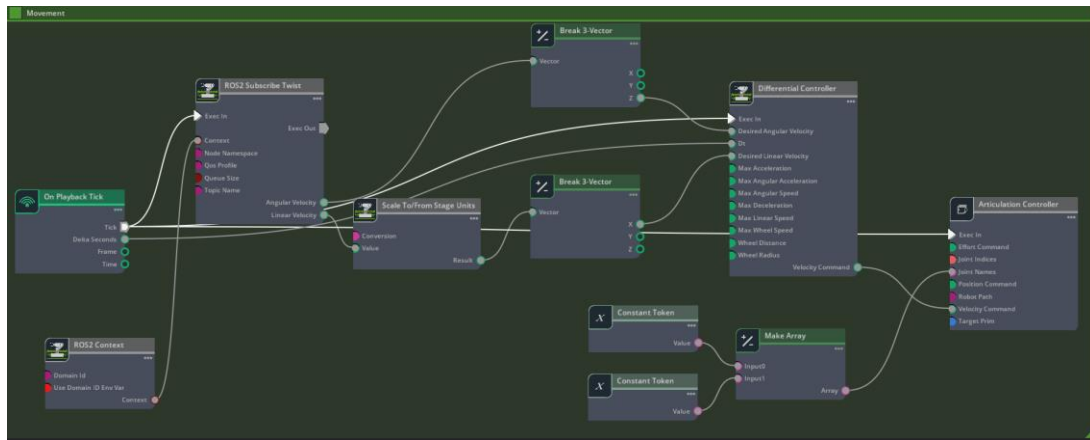
Add the “RTX Lidar” graph with these settings



After adding each graph, go to the 4 different context nodes under them. Uncheck “UseDomainIDEnvVar”, and change “domain_id” to any number that you will use later in the tutorial. For this SOP, I used 67 (hahahaha). This is very important because in order for this computer to communicate with ROS2 on the other linux computer, they must be on the same domain id.



Unfortunately, there is not an action graph shortcut for the movement of the rover so you will have to build it yourself. Follow this tutorial to create the action graph: [Turtlebot Movement ActionGraph](#).

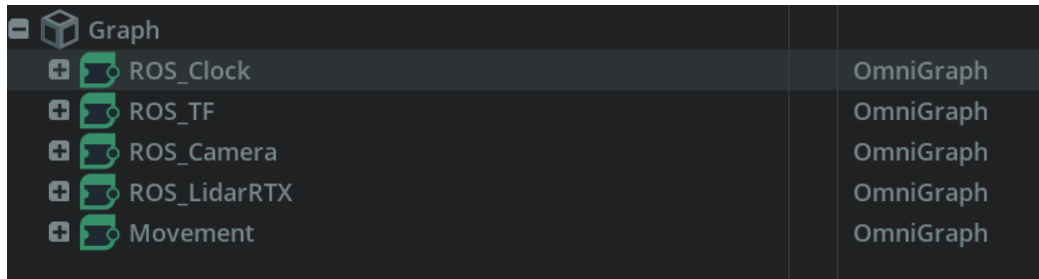


Once you create the action graph, there are a couple of things you will have to change. First change the “context” like the previous graphs. Then change one “constant_token” value to “joint_wheel_left” and the other “constant_token” to “joint_wheel_right”. Next, click on the “differential_controller” node and change it to these settings

▼ Differential Controller Node		
▼ Inputs		
Desired Angular Velocity	0.0	=
dt	0.0	=
Desired Linear Velocity	0.0	=
maxAcceleration	0.0	-
maxAngularAcceleration	0.0	-
maxAngularSpeed	2.0	+
maxDeceleration	0.0	-
maxLinearSpeed	1.67	+
maxWheelSpeed	0.0	-
wheelDistance	0.4132	+
wheelRadius	0.1	+

Lastly, go to the “articulation_controller” node and change “target_prim” to “nova_carter_base”. Once you are done, your graph page on “Stage” should look

like this



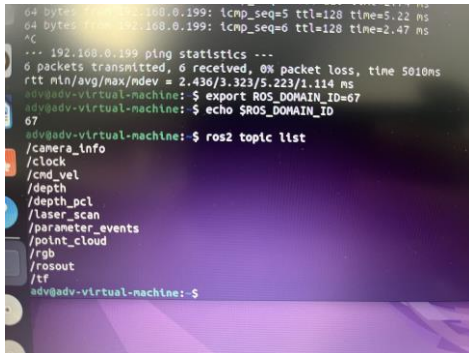
6) The next part of the tutorial will be on the Linux computer with ROS2.

If you don't have anything installed on your other computer, make sure it is on the Ubuntu 22.04 operating system and download the basic version of ROS2 **Humble**.

The first step will be to connect both individual computers to the same LAN. This is essential as it is the way that the computers, ROS2, and Isaac Sim will communicate with each other. You can check the IP on windows by pressing "windows + r", typing in "cmd" and pressing enter, then typing "ipconfig" in the terminal. For your ubuntu computer, you check by opening a terminal and typing in "ip a". You can confirm that they are connected to the same LAN and subnet if the first numbers are similar. Ex) 192.168.0.199 and 192.168.0.155. To verify that they can communicate with each other, type in "ping (Ip of windows computer)" in your ubuntu terminal and it should constantly ping it until you press "Ctrl + C" to stop it.

The next step will be to change the domain id of your ubuntu computer to the same one you set previously. To do this you type in "export ROS_DOMAIN_ID=67" on your ubuntu computer. To verify, type in "echo \$ROS_DOMAIN_ID" and it should return 67. Once you have confirmed it, your Isaac Sim program and ROS2 should be able to communicate with each other.

To confirm this, first press the **Triangle play button** on Isaac Sim. After this, type in “ros2 topic list” on ubuntu and the topic list should look like this



```
64 bytes from 192.168.0.199: icmp_seq=5 ttl=128 time=5.22 ms
64 bytes from 192.168.0.199: icmp_seq=6 ttl=128 time=2.47 ms
^C
... 192.168.0.199 ping statistics ...
6 packets transmitted, 6 received, 0% packet loss, time 5010ms
rtt min/avg/max/mdev = 2.436/3.323/5.223/1.114 ms
adv@adv-virtual-machine: $ export ROS_DOMAIN_ID=67
adv@adv-virtual-machine: $ echo $ROS_DOMAIN_ID
67
adv@adv-virtual-machine: $ ros2 topic list
/camera_info
/clock
/cmd_vel
/depth
/depth_pcl
/laser_scan
/parameter_events
/point_cloud
/rgb
/rosout
/tf
adv@adv-virtual-machine: $
```

- 7) This part of the tutorial will show you the different ways you can interact with Isaac Sim using ROS2 commands, and the software called Rviz2

The first thing we will try is sending the rover a “cmd_vel” command. This is essentially a command that tells the rover what velocity to go. First, **make sure the simulation in Isaac Sim is playing**. After this, you can type in this command on the ubuntu terminal: `ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"`

This should cause the rover to start moving in the simulation.

The next thing we will learn to use is the software called Rviz2. This software showcases the output of the camera and lidar on the rover. To enter the software type in “rviz2” in your terminal. The first thing you want to when entering the software is changing “Fixed Frame” on the leftside from “map” to “chassis_link”. After this press “add -> By topic -> /rgb -> Image”. By doing this, you should be able to see the rover's camera which shows the rubiks cube. To verify that the lidar is working, we press “add -> By topic -> /point_cloud -> PointCloud2” and “add -> By topic -> /laser_scan -> LaserScan”. If it shows the Lidar rotating around the middle of the map, it means its working

- 8) This last part of the SOP will show putting the AMR in a virtual environment, scanning the environment, and running.

To do this let's create a new stage by “file -> new” and adding an example environment by going to the bottom left, typing in “simple_room” in the “Isaac Sim Assets” search bar, and pressing “open file”. Next lets copy the “nova_carter_base” and “graph” from the “stage” of our original file, and paste it into the root of this new

environment. After this, the rover should be sitting on top of the table



Let's move the rover to the ground by selecting “nova_carter_base” from “stage” and using the axis controls on the screen.

The next thing we will do is add a wall in front of our rover. This will be to showcase that the lidar is working and to show the rover running into a physical object. Do this by “create -> mesh -> cube” and placing it in front of the rover. I then change “x: 0.2” and “y:1.3” on “scale” in “properties”. The next step is to add actual physics property to this wall. This will cause it to be affected by gravity and make it a barricade that blocks the rover when it runs into it. Do this by selecting the cube, pressing the green “+add” button in “properties” and going to “physics -> colliders preset”. Then scroll down to physics and check “Collision_Enabled”.

The next thing we will need to do is update some variables on our graphs. This is because our rover is now in the “root” so we have to update it.

First go to the “ROS TF” action graph and go under “PublisherTF” and change “targetPrims” to “/Root/nova_carter_base”.

Next go to the "ROS_camera" action graph and go under “RenderProduct” and change “CameraPrim” to “/Root/nova_carter_base/chassis_link/RSD455/Camera_Psuedo_Depth”.

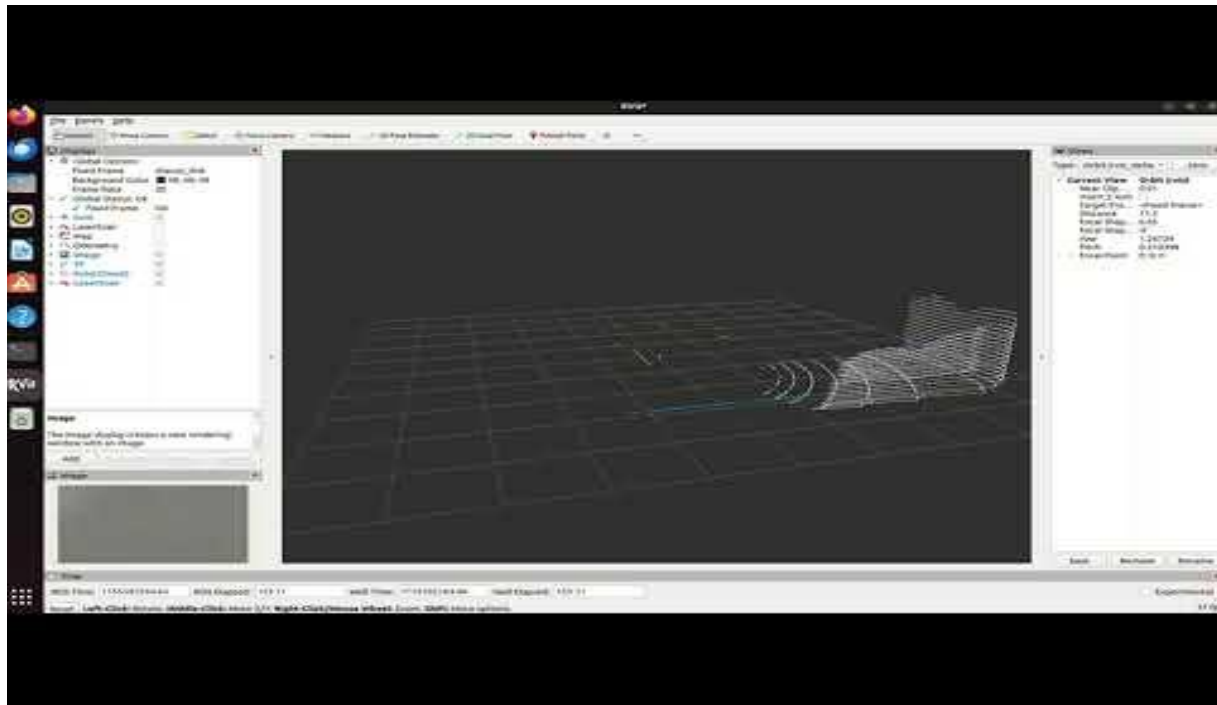
Next go to the lidar action graph, go to render product, and change the target prim to “Root/nova_carter_base/chassis_link/XT_32/PandarXT_32_10hz”.

Lastly, go to the movement action graph, go to “articulation_controller” and change “targetPrim” to “/Root/nova_carter_base”

After changing these settings, we should be able to run our Isaac Sim simulation using ROS2. First enter Rviz2, change fixed frame to “chassis_link”, add the rgb image, pointcloud2, and laserscan. Also press “add -> By display type -> TF” This will show the center axis of your rover. To make it less cluttered, extend it, go to frames, and uncheck everything except “chassis link”

Now we can finally try the simulation! Play the simulation in Isaac Sim, enter Rviz2 and configure everything, and go in another terminal (reset domain id to 67) and type in your cmd_vel command! You should be able to see the rover run into the wall on the Isaac Simulation. On Rviz, you should be able to see the lidar scanning of the wall in front of it, the table besides it, and the room walls surrounding it. Here is an example of how it turned out for me (Press on the picture to play the video)





If you have any urgent questions that chatgpt can't answer,
please reach me at

Achila@ttu.edu

Or

Adrianchila@icloud.com

Thank you!!!!